Iterative programming

Iterative programming

The concept of iteration underlies any operation in programming languages. In R many operations are vectorized. This makes the language more efficient and cleaner BUT hides the concept of iteration.

For example, the `sum()` function lets us add up a vector of numbers.

```
# add up the numbers from 1 to 10
sum(1:10)
```

```
[1] 55
```

But what is hidden underneath?

Example: if I ask you to use the **`print()`** function to write “hello world” in the console 5 times, how would you do it?

```
msg <- "hello world"
print(msg)
```

```
[1] "hello world"
```

```
print(msg)
```

```
[1] "hello world"
```

```
print(msg)
```

```
[1] "hello world"
```

```
print(msg)
```

```
[1] "hello world"
```

```
print(msg)
```

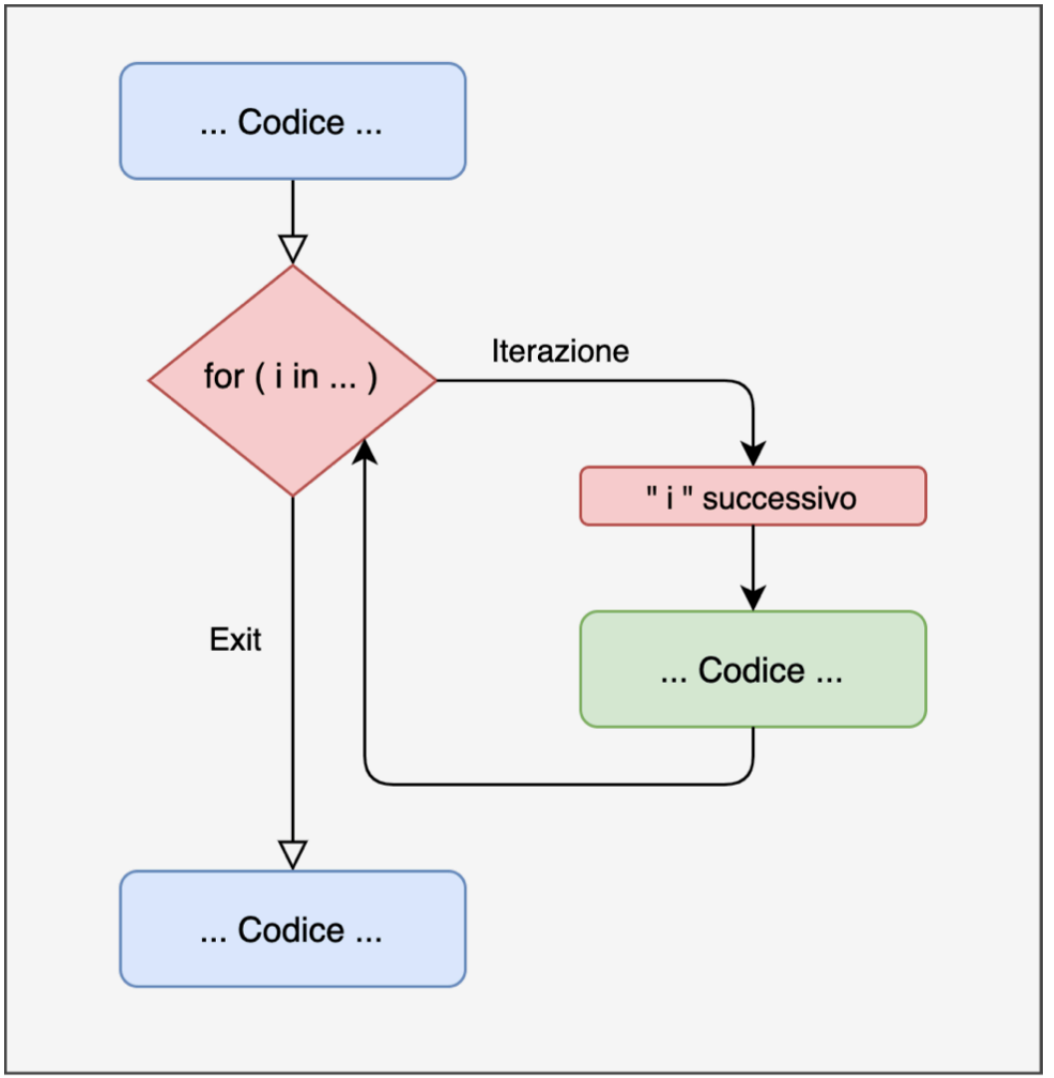
```
[1] "hello world"
```

What we are missing (and what the `sum()` function does) is a way to repeat a given operation without actually repeating the code manually.

There are several constructs that let us repeat operations; the most used are:

- The `for` loop
- The `while` loop
- The `repeat` loop
- The `*apply` family

The `for` loop



The `for` loop is a structure that lets us repeat a portion of code a finite, pre-determined number of times.

A for loop is written as:

```
n <- 10
for(i in 1:n){
# which operations
}
```

If I want to print something 5 times, I can simply use a for loop:

```
for(i in 1:5){
print(paste("hello world", i, sep = "_"))
}
```

```
[1] "hello world_1"
[1] "hello world_2"
[1] "hello world_3"
```

```
[1] "hello world_4"
[1] "hello world_5"
```

Breaking down the `for` loop

- `for(){ }`: this is the implementation in R (similar to the if statement)
- `i`: this is called the iterator or index. It is a generic index that can take any value and name. By convention it is called i, j etc. It keeps track of the number of iterations our loop has to do
- `in <values>`: this indicates the values the iterator will take inside the loop
- `{ # operations }`: these are the operations the loop must perform

The power of the for loop lies in the fact that the **iterator** i takes the values of the vector specified after `in`, one at a time:

```
for(i in 1:10){
  print(i)
}
```

```
[1] 1
[1] 2
[1] 3
[1] 4
[1] 5
[1] 6
[1] 7
[1] 8
[1] 9
[1] 10
```

`For` with an index vs without

`i` represents the index of the elements of the vector vec

```
1 vec <- 1:5
2
3 for(i in 1:length(vec)){
4
5   print(vec[i])
6
7   }
```

```
[1] 1
[1] 2
[1] 3
[1] 4
[1] 5
```

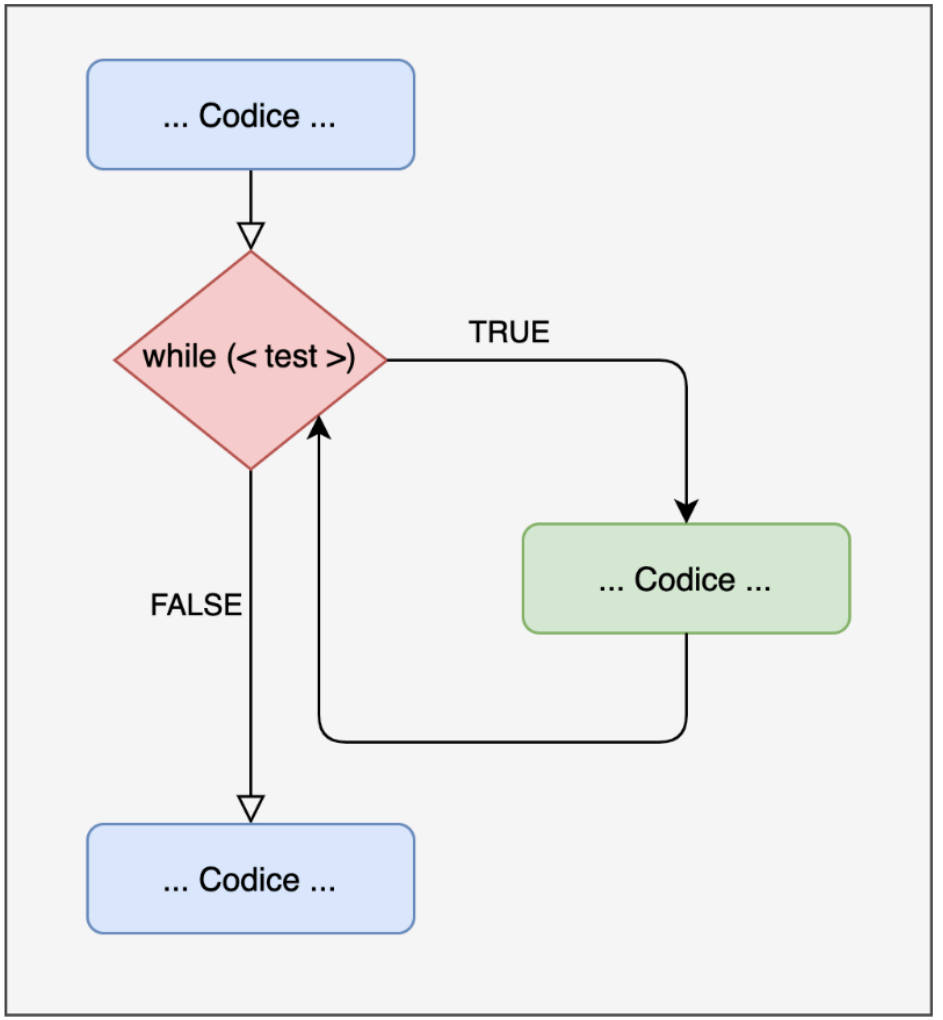
`i` directly represents each element of the vector vec

```
1 vec <- 1:5
2
3 for(i in vec){
4
5   print(i)
6
7 }
```

```
[1] 1
[1] 2
[1] 3
[1] 4
[1] 5
```

The `while` loop

The `while` loop uses a logical condition instead of an iterator and a range of values as in the for loop. The loop keeps going as long as the condition is `TRUE`:



Try writing this `while` loop, see what happens and understand why

```
x <- 10

while (x < 15) {

  print(x)

}
```

The `while` loop is not pre-determined, so it always needs a way to be stopped, by making the condition false.

```r
x <- 5

while (x < 15) {
  print(x)
  x <- x + 1
}
```

```
[1] 5
[1] 6
[1] 7
[1] 8
[1] 9
[1] 10
[1] 11
[1] 12
```

```
[1] 13
```

The `repeat` loop

The logic of the `repeat` loop is very similar to that of the `while` loop, but with 3 important differences:

1. it always performs at least one iteration
2. it emphasizes repetition until a condition is met
3. it uses the `break` command to stop

The `repeat` loop

Roll the die until a 6 comes up:

```
repeat{
  die_roll <- sample(1:6, 1)
  print(die_roll)
  if(die_roll == 6) break
}
```

```
[1] 3
[1] 5
[1] 4
[1] 3
[1] 4
[1] 5
[1] 2
[1] 6
```

`repeat` vs. `while`

`repeat` evaluates the condition at the **end** of the iteration, while `while` does it at the **beginning**. If the condition is not `TRUE` at the start, while does not run, whereas repeat does:

```
1 i <- 1
2 repeat {
3   print(i)
4   i <- i+1
5   if(i > 3)break
6   }
```

```
[1] 1
[1] 2
[1] 3
```

```
1 i <- 1
2 while (i < 4) {
3   print(i)
4   i <- i+1
5   }
```

```
[1] 1
[1] 2
[1] 3
```

Example: the `sum()` function

Imagine we did not have the `sum()` function and wanted to recreate it: how would we do it? Any ideas?

Sum as iteration

Let’s break the sum down conceptually; we add up the numbers from 1 to 10:

- I take the first one and add it to the second (total = 1 + 2)
- I take the total and add it to the 3rd element: total = total + 3 …

In practice we have:

- our vector to sum
- a total object that progressively accumulates the previous sums

Sum as iteration

```r
total <- 0 # initialize the total to 0
x <- 1:10 # vector x

for(i in 1:length(x)){ # for the whole length of x = 1:10
  total <- total + x[i]
}
```

Let’s put everything inside a function (i.e., I create a function that replicates what the sum function does)

```r
my_sum <- function(x){

  total <- 0 # initialize the total to 0

  for(i in seq_along(x)){  # equivalent to 1:length(x)

    total <- total + x[i]

    }

  return(total)

}
```

```
[1] -13.27224
```

```r
sum(x)
```

```
[1] -13.27224
```

Iteration, applications

The `for` loop is also useful, for example, to simulate data. Suppose we have a variable y (income) that depends on age:

1. I load the dataset I created:

```
1 mydf <- read.csv(file = "data/mydf_2.csv") # load the data
2
3 str(mydf)
```

```
'data.frame':   30 obs. of  4 variables:
 $ id     : int  1 2 3 4 5 6 7 8 9 10 ...
 $ age    : int  29 18 27 40 30 38 42 36 22 45 ...
 $ age_cat: chr  "young" "adolescent" "young" "adult" ...
 $ age_z  : num  -0.1876 -1.1701 -0.3662 0.795 -0.0983 ...
```

2. With a `for` loop, I simulate the variable **y** (income) assuming it depends on **age_z** (age):

```r
set.seed(111) # for reproducibility

niter <- 1000
res_all <- rep(NA, niter)     # all estimated effects
p_val <- rep(NA, niter)       # all p-values

# Observed data
age_z <- mydf$age_z
eff_age <- .2 # age effect
n_obs <- length(age_z)

for(i in 1:niter) {
  y <- 0 + eff_age * age_z + rnorm(n_obs, 0, 1) # always a
```

Result:

```r
# Keep only significant results
res_sig <- res_all[p_val <= 0.05]

# Descriptive statistics
cat("Proportion significant = Power:", mean(p_val <= 0.05))
```

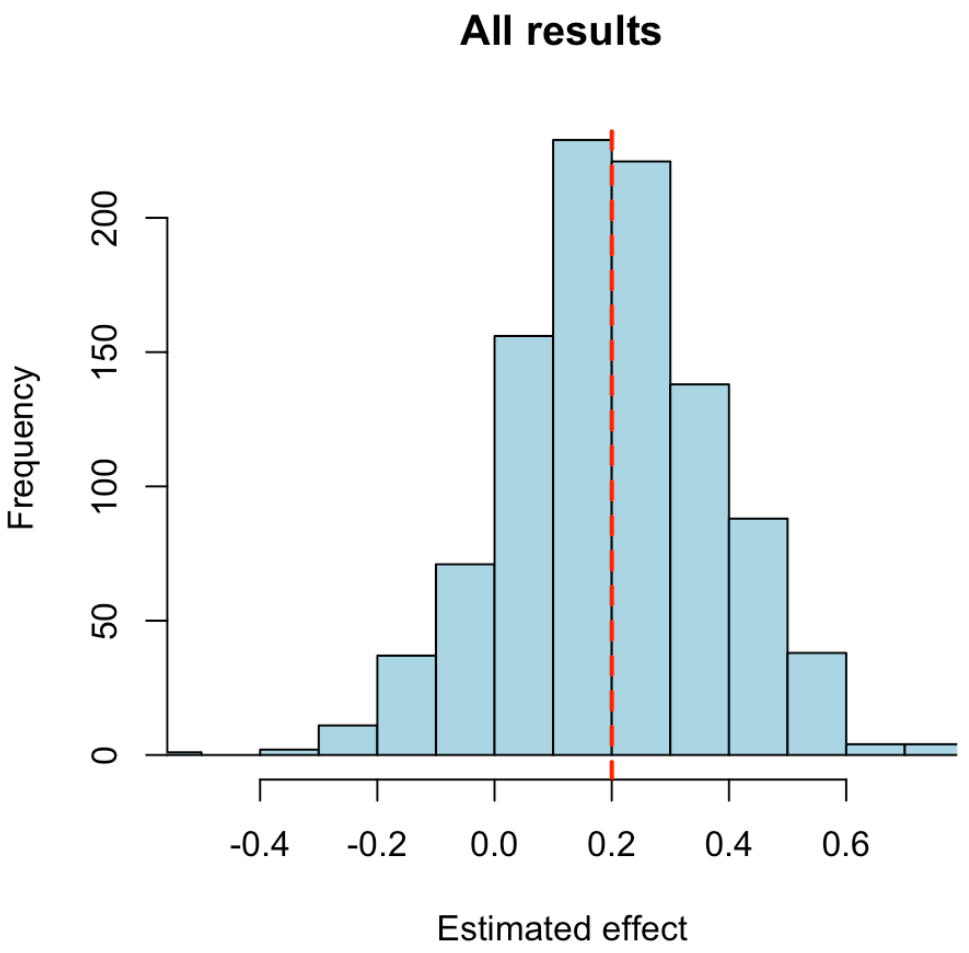
```
Proportion significant = Power: 0.172
```

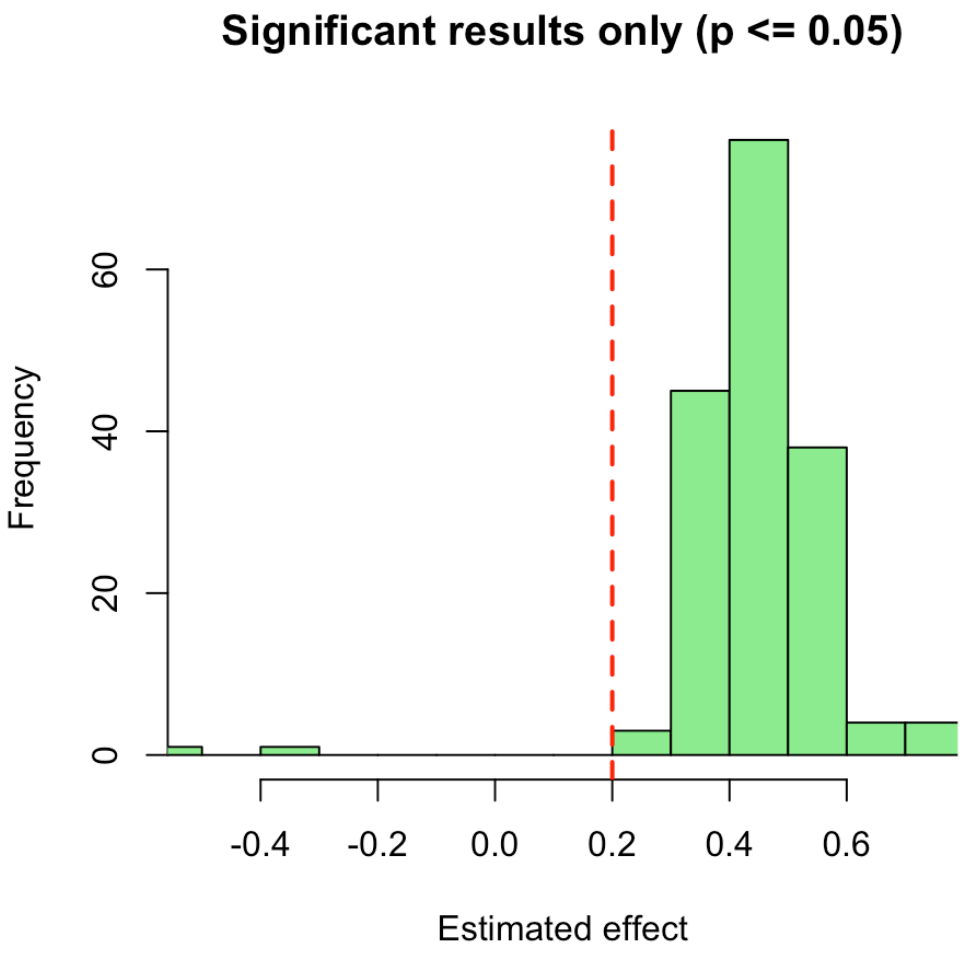
```r
cat("Mean of all effects:", mean(res_all), "\n")
```

```
Mean of all effects: 0.2004597
```

```r
cat("Mean of significant effects:", mean(res_sig), "\n")
```

```
Mean of significant effects: 0.4471571
```


All results
Frequency
0
50
100
150
200
-0.4
-0.2
0.0
0.2
0.4
0.6
Estimated effect


Significant results only (p <= 0.05)
Frequency
0
20
40
60
-0.4
-0.2
0.0
0.2
0.4
0.6
Estimated effect

It is precisely thanks to the for loop that we can run a ***power simulation***.

To learn more, have a look at this page! :)

Nested `for` loops

Once you understand the iterative structure, you can easily extend it by putting a loop inside another one:

```
for(i in 1:3){ # level 1

  for(j in 1:3){ # level 2

      print(paste(i, j))

  }

}
```

```
[1] "1 1"
[1] "1 2"
[1] "1 3"
[1] "2 1"
```

```
[1] "2 2"
[1] "2 3"
[1] "3 1"
[1] "3 2"
[1] "3 3"
```

Imagine we have two experimental groups (for example, a Control group and a Treatment group), each with 10 participants. We want to measure a variable (for example, reaction time) for each participant and record the results in a dataframe:

```r
# Define the study parameters
groups <- c("Control", "Treatment") # Group names
n_subj <- 10                        # Number of subjects
tot_subj <- n_subj * length(groups)

# Create a data.frame to store the data
res <- data.frame(

  # Group factor
  Group = factor(x = rep(groups, each = n_subj)),

  # Subject factor
  Subject = factor(x = 1:tot_subj),
```

```r
head(res, 5) # look at the first observations of res
```

```
    Group Subject RT
1 Control       1  0
2 Control       2  0
3 Control       3  0
4 Control       4  0
5 Control       5  0
```

```r
tail(res, 5) # look at the last observations of res
```

```
        Group Subject RT
16 Treatment      16  0
17 Treatment      17  0
18 Treatment      18  0
19 Treatment      19  0
20 Treatment      20  0
```

```r
# Nested loop to simulate data collection
set.seed(42)  # For reproducibility of the simulated data

k <- 1 # reaction time iterator

# how many levels does Group have?
for (i in 1:length(levels(res$Group))) {

  # start with the first group, which is the control group
  group <- levels(res$Group)[i]

  for (j in 1:n_subj) { # n_subj for each group

```

```r
str(res)
```

```
'data.frame':   20 obs. of  3 variables:
 $ Group  : Factor w/ 2 levels "Control","Treatment": 1 1 1 1 1 1 1 1 1 1 ...
 $ Subject: Factor w/ 20 levels "1","2","3","4",..: 1 2 3 4 5 6 7 8 9 10 ...
 $ RT     : num  0.535 0.486 0.509 0.516 0.51 ...
```

```r
head(res, 3)
```

```
    Group Subject        RT
1 Control       1 0.5354760
2 Control       2 0.4860800
3 Control       3 0.5091611
```

```r
tail(res, 3)
```

```
       Group Subject        RT
18 Treatment      18 0.3940291
```

```
19 Treatment       19 0.3983075
20 Treatment       20 0.4807047
```

```r
mean(res$RT[res$Group == "Control"])
```

```
[1] 0.5142776
```

```r
mean(res$RT[res$Group == "Treatment"])
```

```
[1] 0.4476666
```

Now let’s practice!

Open this link and keep it open:

https://etherpad.wikimedia.org/p/arca-corsoR